WEB APPLICATION VULNERABILITY ASSESSMENT & PENETRATION TESTING

WEEK 7 PRACTICAL REPORT | Burp Suite, Gobuster, OWASP ZAP

Prepared by: Amar Pratap

Environment: Kali Linux in VMware
Target: Damn Vulnerable Web Application (DVWA)
Target URL: http://127.0.0.1:42001/
Assessment date: 20 October 2026
Internship: Cyber Security | DG Interns Hub

CONFIDENTIALITY NOTE: This report documents authorized testing against a local intentionally vulnerable DVWA instance only. It is a training report, not an assessment of a public or production website.

1. Executive Summary

This Week 7 practical focused on web application testing using Burp Suite Community Edition, Gobuster, and OWASP ZAP. The screenshots show that Burp captured requests to the local DVWA target, the application was explored through its pages, a Gobuster directory enumeration completed, and a Reflected Cross-Site Scripting (XSS) test triggered a browser alert displaying “1”.

Confirmed result: Reflected XSS was demonstrated in the local lab. Other observations—including directory/path discovery, login redirects, a failed brute-force page login, and ZAP alert indicators—are recorded as observations or leads, not automatically classified as confirmed vulnerabilities. The supplied evidence does not yet establish three distinct working vulnerabilities.

Assessment at a glance

- Target remained local: 127.0.0.1:42001.
- Burp Proxy listener shown as port 8080.
- XSS proof: browser alert displaying 1 after a script payload.
- Gobuster completed 4,613/4,613 requests and reported paths.
- No verified SQL injection, authentication bypass, or successful brute-force result in the supplied screenshots.

2. Scope, Authorization and Rules

In scope

- Local DVWA instance at http://127.0.0.1:42001/.
- Manual browsing and request inspection with Burp Suite.
- Reflected XSS validation in DVWA.
- Directory/path enumeration using Gobuster and the local common.txt wordlist.
- Passive/manual exploration through OWASP ZAP.

Out of scope

- Any public website, third-party host, or production service.
- Destructive tests, denial-of-service, data extraction, or high-volume password attacks.
- Treating tool alerts or discovered paths as confirmed vulnerabilities without validation.

All testing described here is limited to the intentionally vulnerable lab target. Keep the target local and do not reuse these payloads against systems without explicit written permission.

3. Objectives and Tools

- Intercept and inspect HTTP requests/responses in Burp Suite.
- Map application routes and identify forms/parameters.
- Test reflected input handling for XSS.
- Enumerate common paths with Gobuster.
- Explore requests and alerts in OWASP ZAP.
- Document evidence, impact, limitations, and remediation.

Tools visible in evidence: Burp Suite Community Edition v2026.8, Gobuster v3.8.2, OWASP ZAP v2.17.0, Kali Linux, and DVWA.

4. Test Environment and Methodology

Environment

- Operating system: Kali Linux running in VMware.
- Application: DVWA served locally at 127.0.0.1:42001.
- Burp browser traffic routed through local proxy listener on port 8080.
- Gobuster wordlist: /usr/share/wordlists/dirb/common.txt.
- ZAP local proxy shown on localhost:8081 in the status bar.

Method

The workflow was: (1) open the local application, (2) capture traffic, (3) inspect request methods, paths, status codes, cookies, and redirects, (4) test a controlled reflected input, (5) enumerate common paths, (6) browse through ZAP and inspect alert indicators, and (7) classify each observation according to available evidence.

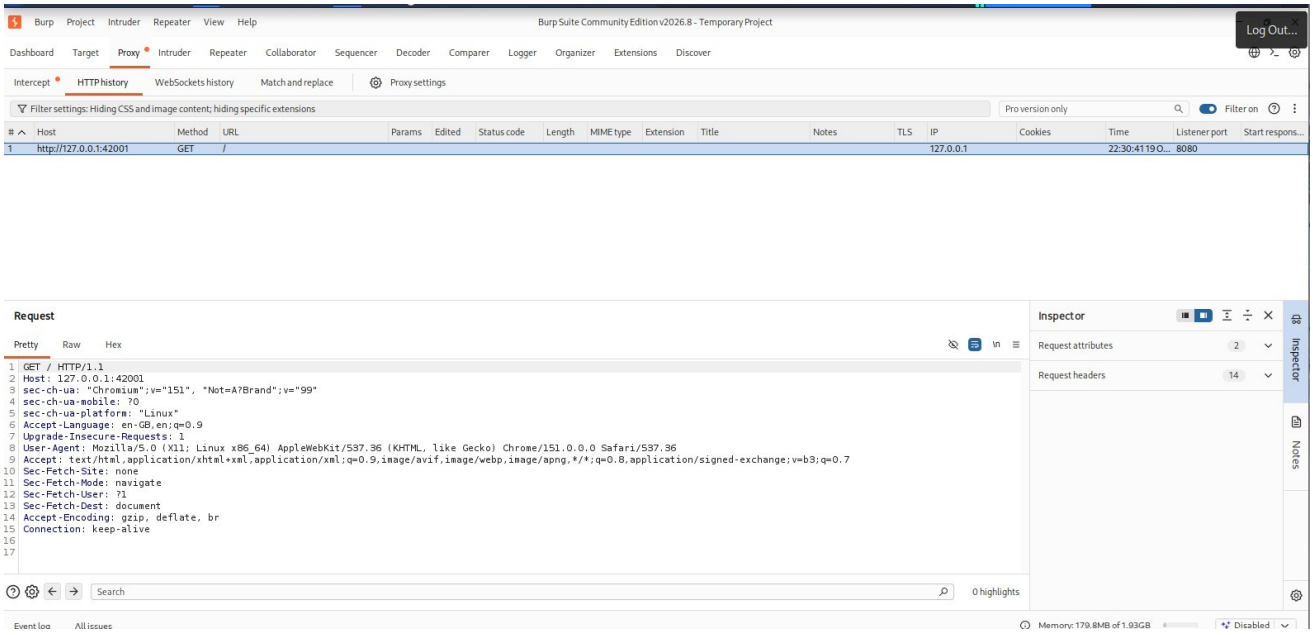


Figure 1. Burp HTTP history showing a request to the local DVWA host.

5. Task 22 — Burp Suite Setup and Request Interception

The Burp screenshots show the Proxy → HTTP history and Intercept views. The local target is 127.0.0.1:42001 and the listener column shows 8080. A GET request to /login.php was intercepted, and the HTTP history includes GET /login.php, POST /login.php, and a root request that returns a redirect to login.php.

What the evidence proves

- Burp is receiving browser traffic from the local DVWA instance.
- The login endpoint is /login.php and both GET and POST methods were observed.
- A 302 Found response with Location: login.php is a redirect; it is not by itself proof of a login bypass or failed login.
- The response includes session-related cookies; cookie presence alone does not prove a session vulnerability.

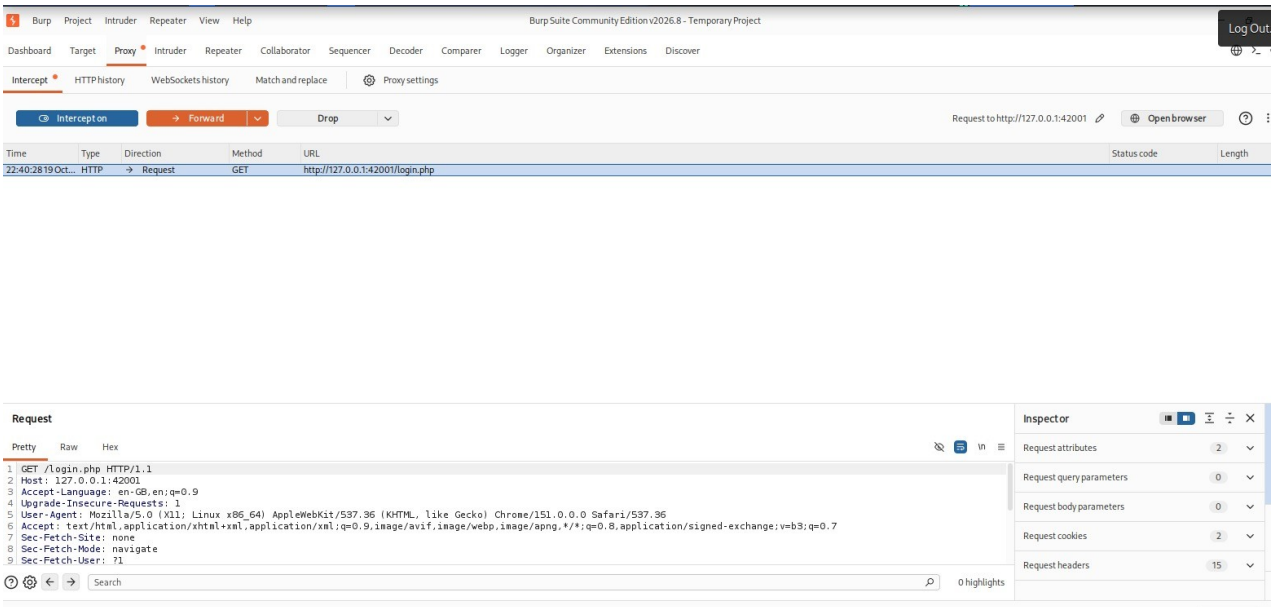


Figure 2. Intercepted GET request to the local login endpoint.

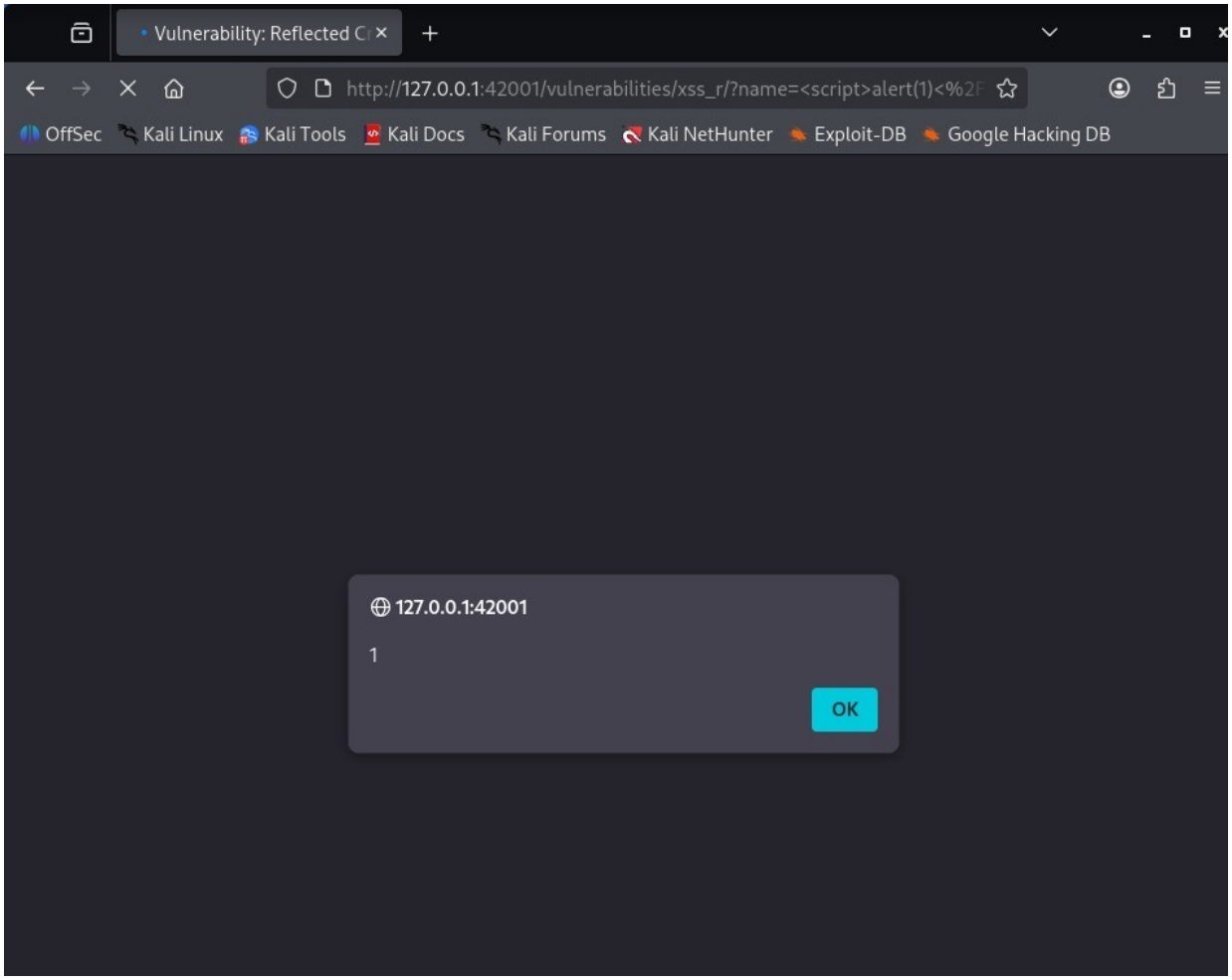


Figure 3. HTTP history response showing 302 Found and Location: login.php.

6. Task 23 — Application Mapping and Attack Surface

The Burp Target → Site map view contains the local host and login endpoint. The HTTP history also shows requests for the home page and login resources. This is useful initial mapping, but the screenshots do not show a complete crawl of every DVWA module or every form parameter.

Observed routes and parameters

- GET / and GET /login.php were observed.
- POST /login.php was observed, but the screenshot does not expose the submitted credentials in a clearly readable request body.
- The Reflected XSS module was reached at /vulnerabilities/xss_r/ with a name query parameter.
- The Brute Force module URL includes username and password query parameters, visible in the browser address bar.

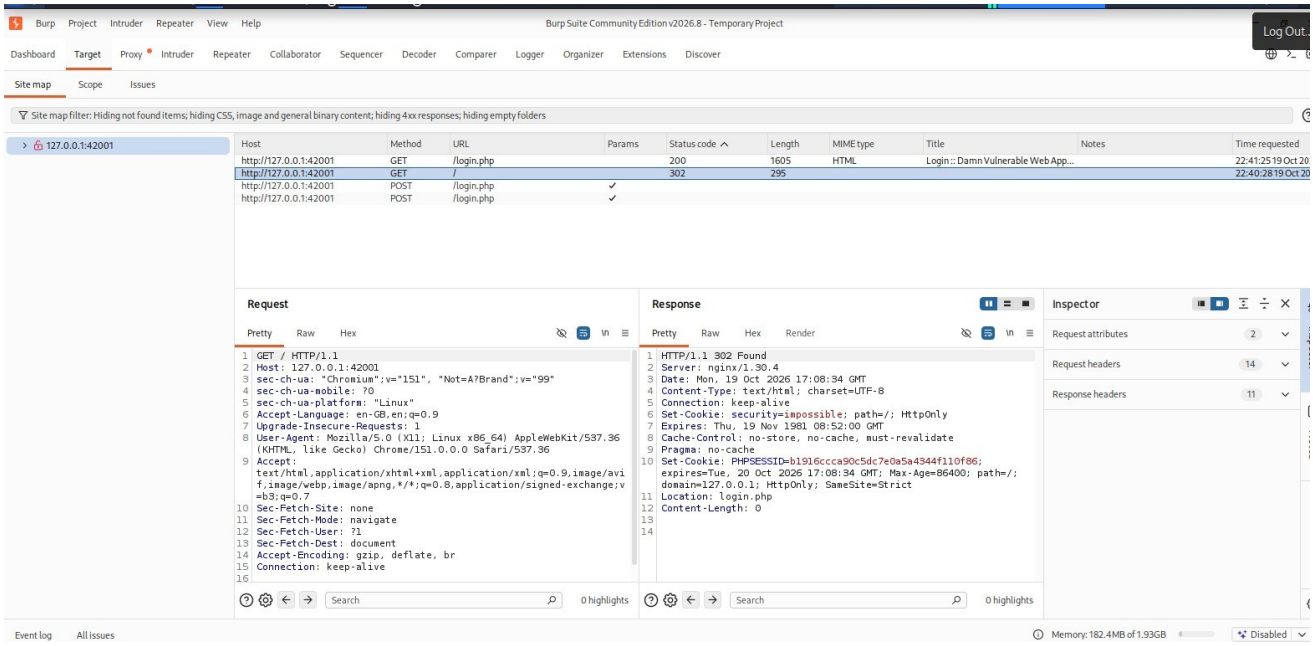


Figure 4. Burp Target site map for the local DVWA host.

Recommendation: for a complete map, browse each in-scope DVWA module manually and record its URL, method, input names, authentication requirement, and response behavior. Keep evidence focused on the local host; the screenshot containing a Google search request is unrelated to the target and should not be presented as a finding.

7. Task 24 — Reflected XSS Test

Test case

Module: DVWA → Vulnerability: Reflected Cross Site Scripting (XSS). Parameter: name. A harmless baseline value “hello123” was submitted first. The page reflected the value as “Hello hello123”. A script payload was then submitted and the browser displayed a JavaScript alert containing the value “1”.

Result: Confirmed in the lab

The alert screenshot is direct evidence that the supplied input was interpreted as executable JavaScript in the browser in this DVWA configuration. This supports a confirmed Reflected XSS finding for the local training application. It does not establish that another application is vulnerable.

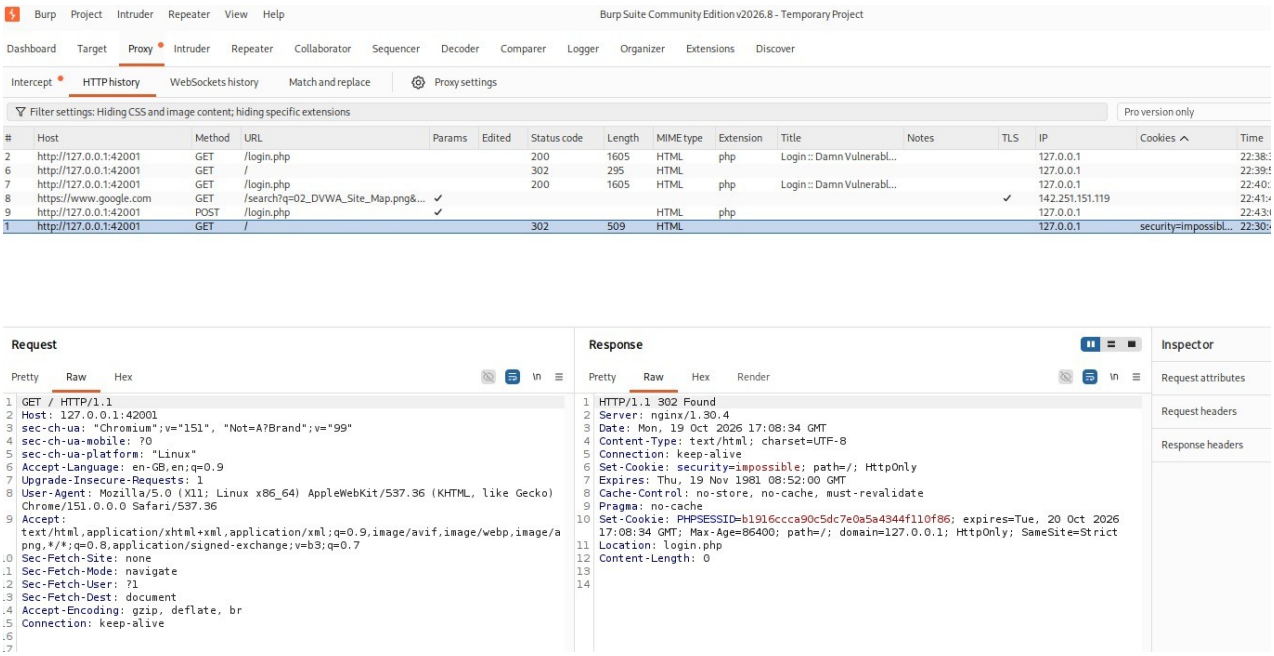


Figure 5. Baseline input reflected by the DVWA page.

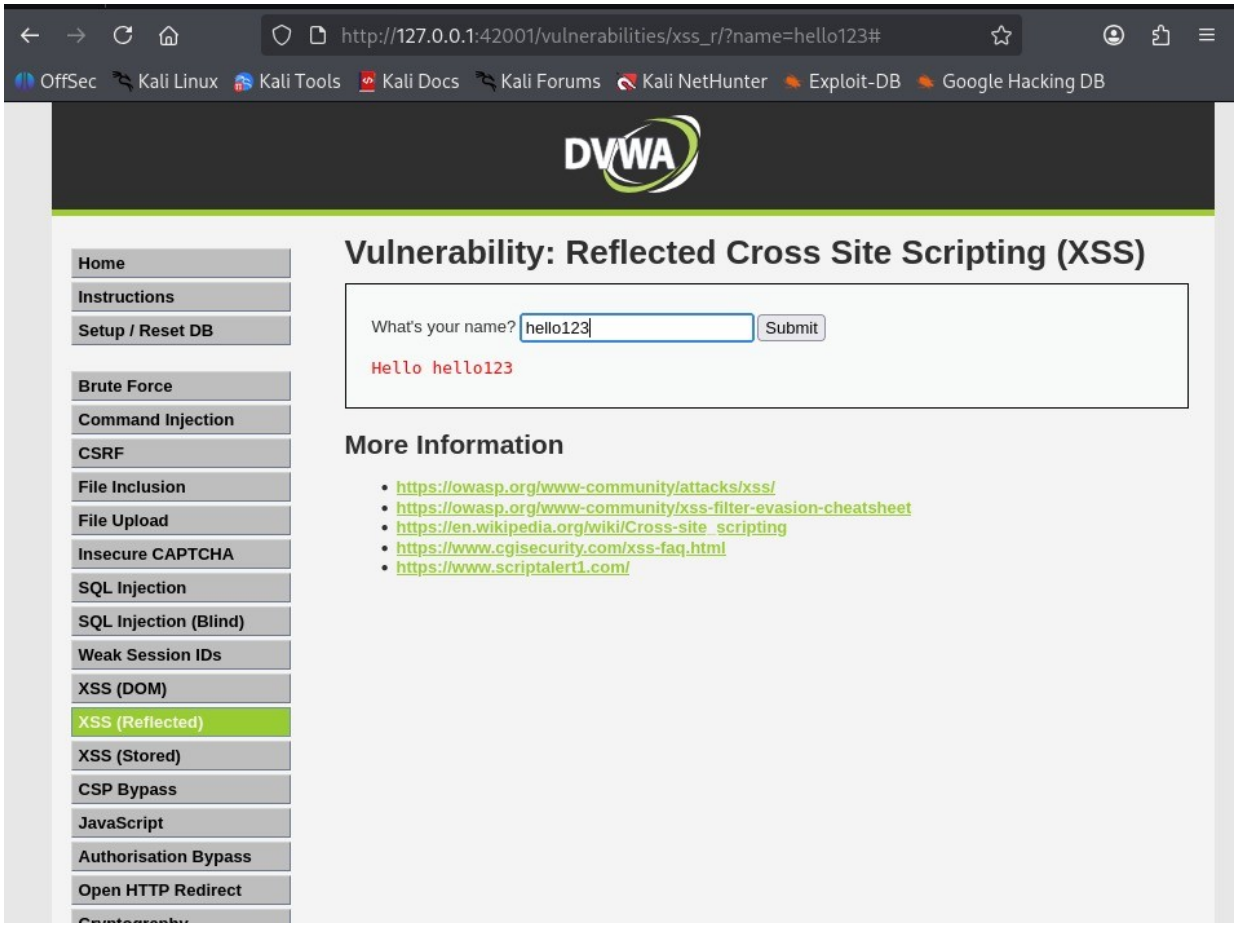


Figure 6. Browser alert displaying “1” after the reflected XSS test.

Impact

In a real application, reflected XSS may allow attacker-controlled script execution in a victim’s browser under the vulnerable site’s origin. Depending on application context, impact can include UI manipulation, phishing, or actions performed in the victim’s authenticated session. Do not claim session-cookie theft from this test; the screenshot does not demonstrate it.

Recommended remediation

- Apply context-aware output encoding to all untrusted data before rendering it in HTML.
- Use safe templating APIs and avoid inserting untrusted values into executable JavaScript or HTML contexts.
- Validate inputs as a secondary control; validation alone is not a replacement for output encoding.
- Deploy a restrictive Content Security Policy as defense in depth.
- Add regression tests for reflected inputs and verify the fix in the same browser context.

8. Task 24 — Authentication / Brute Force Observations

The Brute Force module screenshot shows a login form with “Username and/or password incorrect.” This proves that the particular submitted credentials were rejected. It does not prove successful brute force, weak credentials, authentication bypass, or a rate-limit failure.

Safe validation still needed

- Use only a small number of manually entered test credentials in the local lab.
- Record the exact request and response for a known invalid attempt.
- If testing lockout/rate limits, document the configured DVWA security level and observe whether attempts are delayed or blocked; avoid high-volume automation.
- Verify access control by logging out and attempting to open a protected page, then record whether the app redirects or denies access.

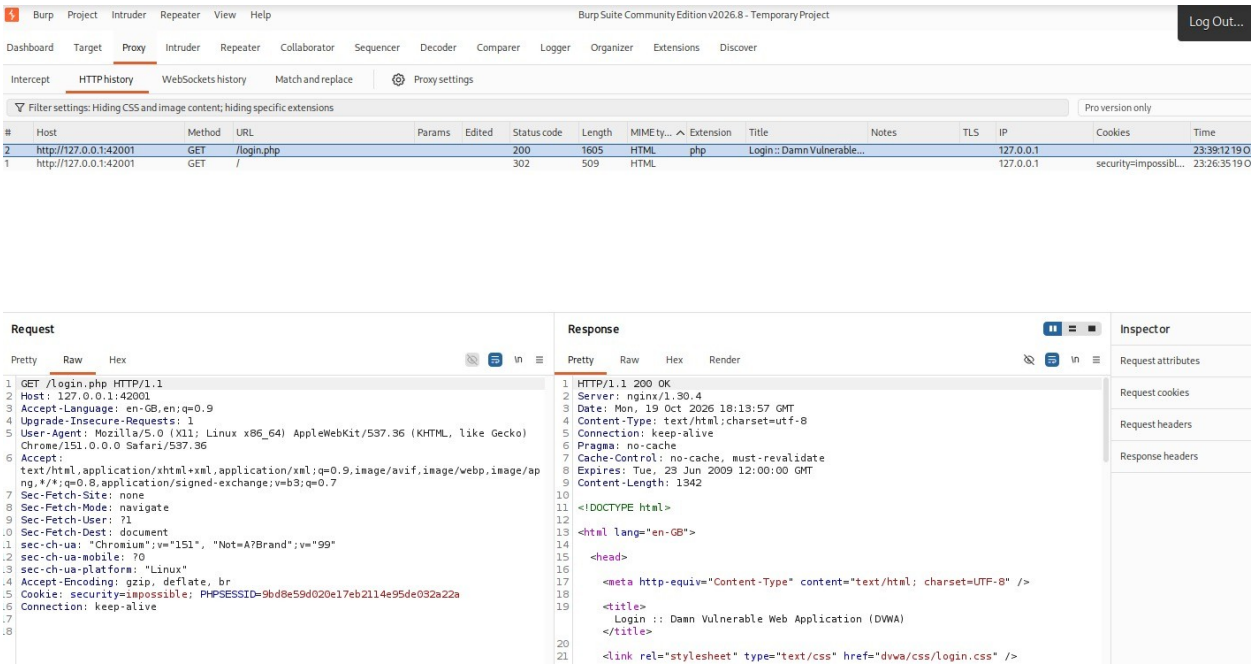


Figure 7. DVWA Brute Force page showing a rejected login attempt.

9. Task 24 — SQL Injection and Parameter Tampering

The provided screenshots do not show a SQL Injection payload with a result that demonstrates altered database behavior, an error, or an authentication bypass. Therefore SQL injection is not reported as confirmed. A proper report would need a baseline request, the exact modified parameter, the corresponding response, and repeatable evidence of changed behavior.

For parameter tampering, compare a normal request with one controlled parameter change in Burp Repeater while remaining on the local DVWA target. Record the security level and result. Do not label a test as vulnerable merely because a request can be edited.

10. Task 25 — Gobuster Directory Enumeration

Gobuster v3.8.2 was run against the local DVWA URL using /usr/share/wordlists/dirb/common.txt. The terminal screenshot shows the scan completed with 4,613 / 4,613 requests (100%). Paths shown include /config/, /database/, /docs/, /external/, /favicon.ico, /index.php, /phpinfo.php, /php.ini, and /robots.txt with responses including 200, 301, and 302.

Interpretation

- A 301 commonly indicates a redirect; follow the redirect and assess what is actually accessible.
- A 302 redirect to login.php does not mean the target file is exposed.
- A 200 response is not sufficient by itself to classify a path as sensitive disclosure; inspect the returned content and access controls.
- Because DVWA is intentionally vulnerable, results should be interpreted as lab observations rather than evidence about a real deployment.

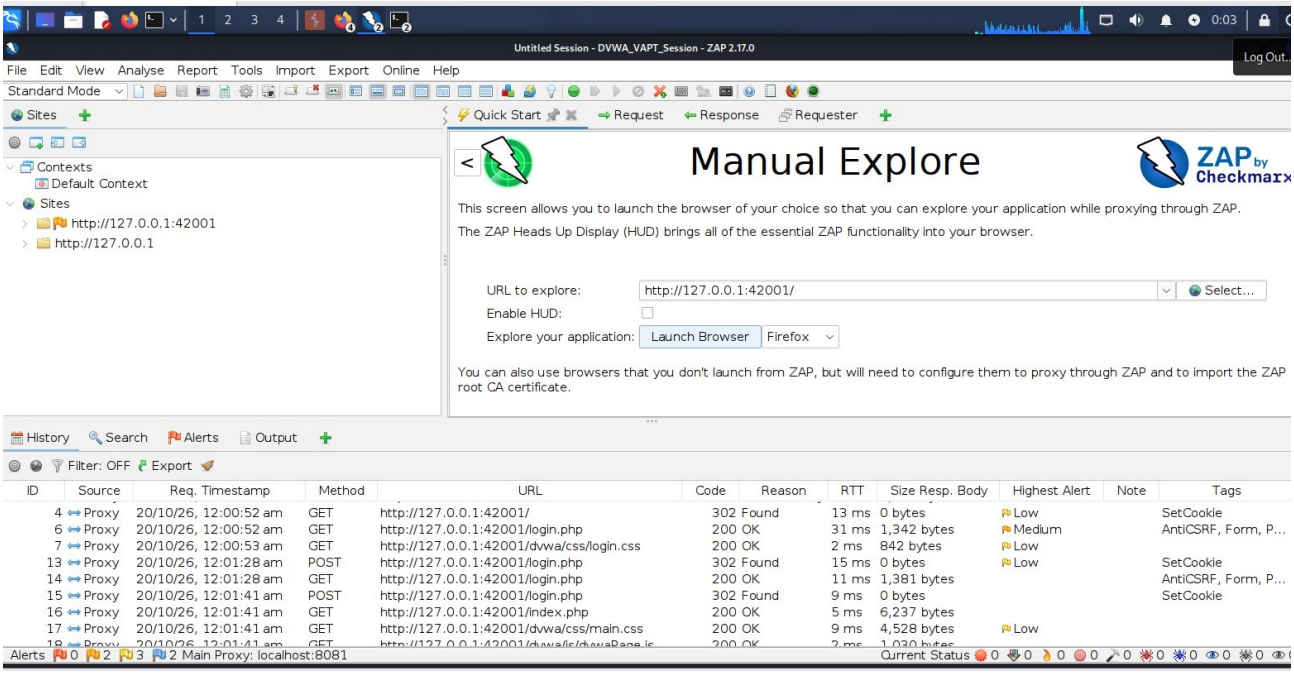


Figure 8. Gobuster completion and discovered paths in the local DVWA lab.

Potential follow-up: manually open only the discovered local paths, inspect whether sensitive content is actually served, and record the response body. Avoid claiming disclosure for /php.ini or /phpinfo.php unless the content itself is accessible and sensitive information is demonstrably exposed.

11. Task 26 — OWASP ZAP Exploration

The ZAP screenshots show version 2.17.0, a Manual Explore URL pointing to the local DVWA target, the local site in the Sites tree, and request history entries. Alert indicators/tags appear in the history table, including items such as SetCookie and AntiCSRF/Form-related tags. These are useful leads but are not enough to state that each alert is a confirmed vulnerability.

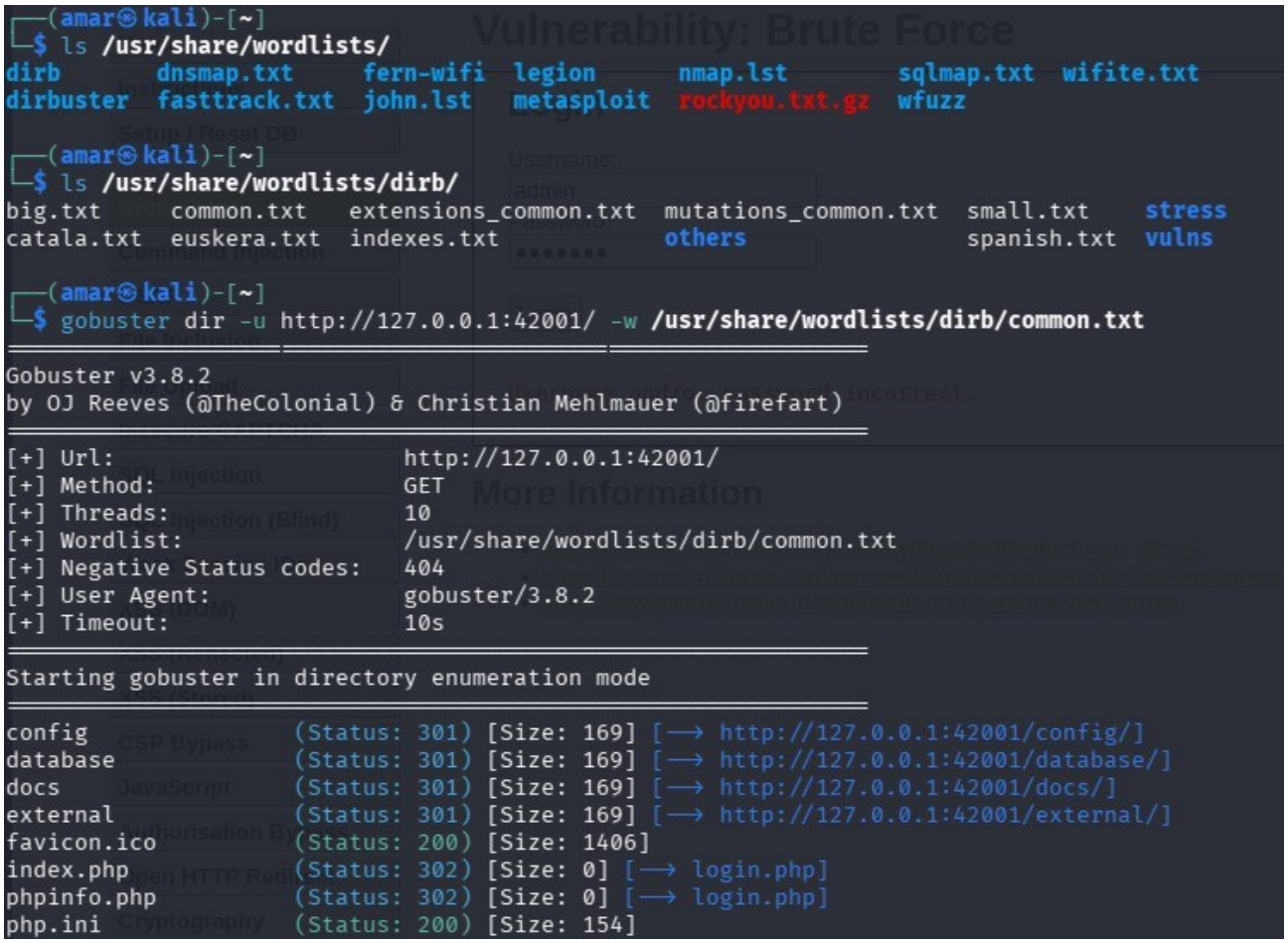


Figure 9. OWASP ZAP session with local DVWA site and request history.

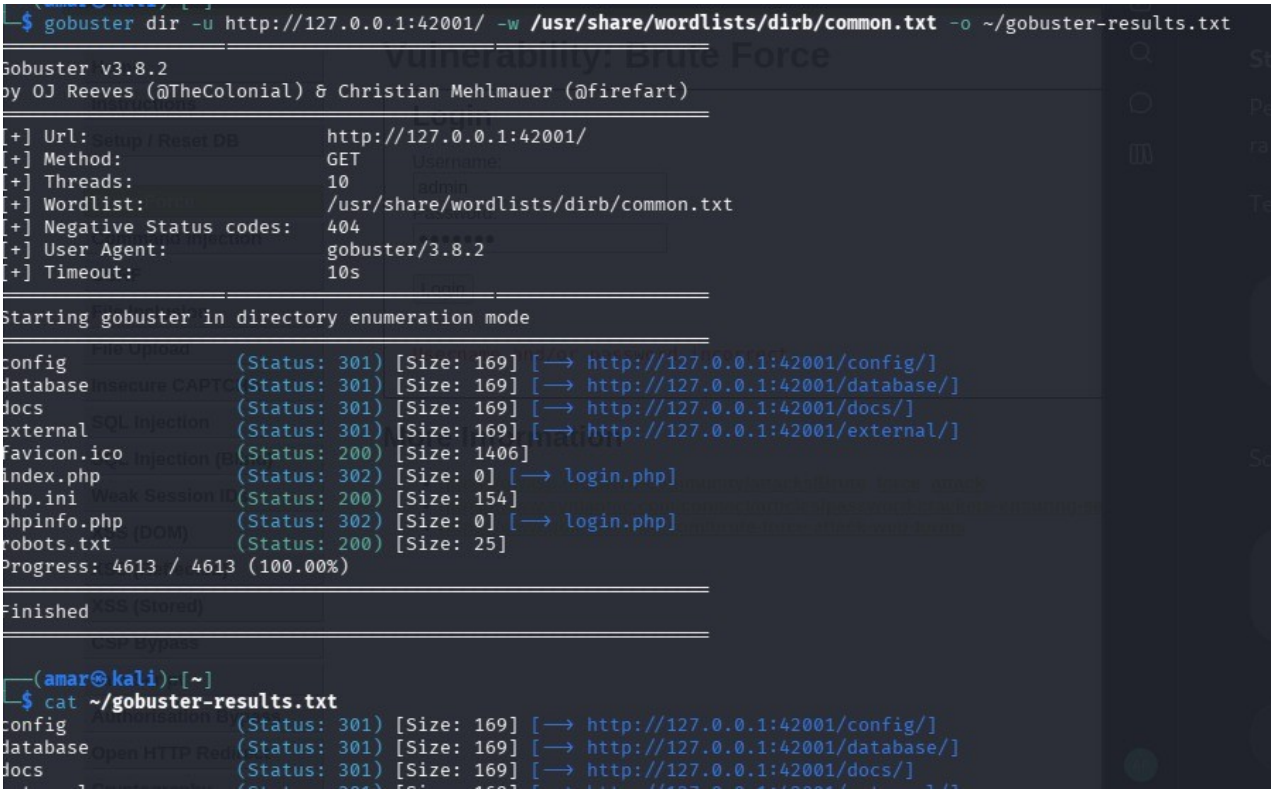


Figure 10. ZAP Manual Explore with the local DVWA browser.

Validation required

- Open each alert and record its exact name, risk, confidence, affected URL, evidence, and solution.
- Manually validate the alert before listing it as a confirmed finding.
- Export a ZAP report only after reviewing alerts and removing irrelevant/false-positive entries.
- Do not equate generic tags such as SetCookie or AntiCSRF with a confirmed exploitable issue.

12. Findings Register

ID	Item	Status	Evidence	Impact / handling
F-01	Reflected Cross-Site Scripting (XSS)	Confirmed in local DVWA	Browser alert displayed “1” after input was reflected and executed.	High potential impact in a real application; actual severity depends on context.
O-01	Common path discovery	Observed; not confirmed vulnerability	Gobuster completed and returned several paths/status codes. Content exposure not established.	Informational until access/content is validated.
O-02	Login redirects and cookies	Observed; not confirmed vulnerability	302 response to login.php and session-related cookies observed.	No vulnerability conclusion from redirect/cookie presence alone.
O-03	Rejected brute-force login	Observed negative test	Page displayed incorrect username/password.	No successful brute force or rate-limit weakness proven.
O-04	ZAP alert indicators	Needs validation	History shows alert tags/indicators, but individual alert details are not supplied.	Do not assign severity until manually verified.
N-01	SQL injection	Not demonstrated by supplied evidence	No reproducible payload/result pair showing SQL behavior change.	Do not report as a finding yet.

13. Evidence Index

- E1 — Burp HTTP history and request details: Figure 1.
- E2 — Burp Intercept login request: Figure 2.
- E3 — Redirect response: Figure 3.
- E4 — Target site map: Figure 4.
- E5 — Reflected XSS baseline: Figure 5.
- E6 — Reflected XSS JavaScript alert: Figure 6.
- E7 — Brute Force rejection: Figure 7.
- E8 — Gobuster completed enumeration: Figure 8.
- E9–E10 — ZAP site/history and Manual Explore: Figures 9–10.

14. Recommendations and Next Steps

- Fix the confirmed XSS behavior using context-aware output encoding and regression tests.
- Complete SQLi validation in the local DVWA environment and capture baseline vs modified request/response evidence.
- Review each ZAP alert individually and keep only manually verified findings.
- Validate whether discovered paths expose content; redirects alone are not proof of exposure.
- Record the DVWA security level, browser, exact time, payload, request/response, and repeatability for each test.
- Add at least two more independently verified findings before claiming the assignment requirement of three working vulnerabilities is satisfied.
- Keep screenshots readable and crop unrelated desktop/browser content when preparing the final submission.

15. Limitations and Conclusion

This report is based on the screenshots supplied for review, not a live re-test of the application. It does not claim that every DVWA module was tested or that all tool alerts are valid. The strongest confirmed result is Reflected XSS in the local DVWA training environment. Burp request interception, basic application mapping, Gobuster enumeration, and ZAP exploration were also demonstrated. SQL injection, authentication bypass, successful brute force, and sensitive-file disclosure remain unverified from the evidence currently available.

Conclusion: the practical workflow is substantially correct, but the final report should be evidence-led. A tool finding or discovered URL should only be called a vulnerability after the behavior and impact are demonstrated and repeatable.

How to read the results

A status code describes the HTTP response, not the security impact by itself. A 200 response means the server returned content; 301 and 302 usually indicate redirects; none of these codes alone proves a vulnerability. A vulnerability should be reported only when the observed behavior, affected input or route, and plausible security impact are supported by reproducible evidence.

Evidence quality and confidence

The strongest evidence in this submission is the Reflected XSS browser alert showing the value 1 after the test input was submitted. Burp request history, the site map, Gobuster output, and ZAP history demonstrate that the tools were used against the local lab. They are useful supporting evidence, but they do not independently establish SQL injection, authentication bypass, successful brute force, or sensitive-file disclosure.

Recommended next validation steps

For each additional finding, capture a baseline request, a single controlled test change, the resulting response, and the visible effect in the browser. Record the DVWA security level and

exact parameter tested. For ZAP, open each alert and note its name, risk, confidence, URL, evidence, and remediation before including it. Retest after remediation and keep the final report aligned with what the screenshots actually prove.

Appendix — Example evidence checklist

[ ] Capture exact request and response for each finding.

[ ] Show payload and resulting behavior in the same evidence set.

[ ] Record risk rationale and realistic impact.

[ ] Add a remediation specific to the root cause.

[ ] Re-test after the fix and document pass/fail.

[ ] Confirm final PDF page count and slide count before submission.